Parte 1 – Seleção, ciclos, métodos

Bruno Leitão, Fernando Barros, Iury Araújo,
Luís Cordeiro, Marília Curado, Nuno Pimenta

Os conteúdos desta apresentação baseiam-se nos materiais produzidos por António José Mendes para a unidade curricular de Programação Orientada a Objetos.

Quaisquer erros introduzidos são da inteira responsabilidade dos autores.

SELEÇÃO

Seleção

- A estrutura de seleção é utilizada para realizar uma sequência de instruções que é determinada de acordo com condições específicas
- Muitas linguagens de programação incluem uma opção de seleção
  - Java, C++, .NET, ….
- A estrutura verifica se uma condição é verdadeira e, nesse caso, executa uma instrução ou bloco de instruções, caso contrário, (i.e., a condição é falsa) o programa continua a sua execução fora da estrutura

```
if (expressaoBooleana) {
    //instruções que serão executadas caso a expressaoBooleana resulte true
}
```

Seleção

- Um bloco de instruções é delimitado por um par de chavetas (ou chaves);
- Caso se pretenda executar apenas uma instrução, as chavetas que delimitam o bloco são opcionais.

```
int nota = 15;
if (nota >= 10){
    System.out.print("Aluno aprovado.");
}
```

```
int nota = 15;
if (nota >= 10)
    System.out.print("Aluno aprovado.");
```

Seleção

- A instrução de seleção dupla executa uma instrução (ou bloco) no caso da expressão booleana de `if` resultar em `true`, e outra (ou bloco) no caso da expressão booleana resultar em `false`

```
int nota = 15;
if (nota >= 10) {
    // instruções se resultar em 'true'
    System.out.print("Aluno aprovado");
} else {
    // instruções se resultar em 'false'
    System.out.print("Aluno reprovado");
}
```

Seleção

- A instrução de seleção podem ser encadeadas (aninhadas), representando uma arvore de seleção mais complexa

```
if (exp1) {
    if (exp2) {
        // Executado caso exp1 e exp 2 resultem em true
    } else {
        // Executado caso exp1 resulte em true, e exp2 em false
    }
}
else {
    if (exp3) {
        // Executado caso a exp1 resulte em false, e exp3 em true
    } else {
        // Executado caso exp1 e exp3 resultem em false
    }
}
```

Seleção

- Exemplo: escolha um número entre 1 e 3

```
Scanner entrada = new Scanner(System.in);
System.out.println("Escolha um número entre 1 e 3");
int numero = entrada.nextInt();
if (numero == 1) {
    System.out.println("O número escolhido foi: 1.");
} else {
    if (numero == 2) {
        System.out.println("O número escolhido foi: 2.");
    } else {
        if (numero == 3) {
            System.out.println("O número escolhido foi: 3.");
        } else {
            System.out.println("O número escolhido é inválido!" +
                               "Digite um número entre 1 e 3.");
        }
    }
}
```

Seleção condicional

- Java fornece o operador condicional (?:) que pode ser usado no lugar de um comando condicional `if-else` em algumas situações
- A vantagem é tornar o código mais compacto e eficiente, e a desvantagem é tornar o programa mais difícil de compreender
- O operador é composto de três operandos
  - Uma expressão booleana
  - Um valor a ser utilizado caso a expressão seja verdadeira
  - O valor a ser utilizado caso a expressão seja falsa

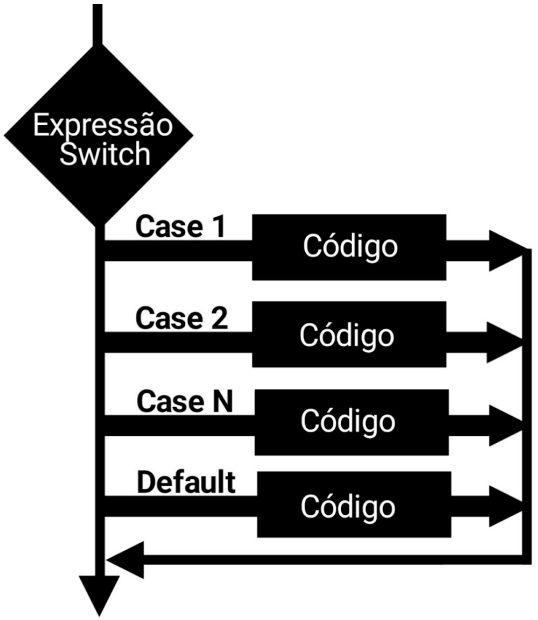
Seleção condicional

```
Scanner sc = new Scanner(System.in);
int x = sc.nextInt();
int y = (x < 0)? x * 10: x / 2;
System.out.printf("x = %d >= y = %d n", x, y);
```

- Se o valor de `x` for menor que zero, `y` receberá o valor de `x` multiplicado por 10
- Se o valor de `x` for maior (ou igual) a zero, `y` receberá o valor de `x` dividido por 2

Switch

- A instrução `switch` por vezes chamada de `switch...case` possibilita a execução condicional de instruções de acordo com a correspondência entre a expressão avaliada e cada constante em `case`



Source: https://blog.betrybe.com/java/switch-case-java/

```
switch (expressao) {
    case constante1:
        // Instruções
        break;
    case constante2:
        // Instruções
        break;
    case default:
        // Instruções
}
```

Switch

- Exemplo: escolha um número entre 1 e 3

```
Scanner entrada = new Scanner(System.in);
System.out.println("Escolha um número entre 1 a 3");
int numero = entrada.nextInt();
 switch (numero) {
   case 1:
     System.out.println("O número escolhido foi: 1.");
     break;
   case 2:
     System.out.println("O número escolhido foi: 2.");
     break;
   case 3:
     System.out.println("O número escolhido foi: 3.");
     break;
  default:
     System.out.println("O número escolhido é inválido! Digite um
      número entre 1 e 3.");
 }
```

Switch (Alternativa)

```
int day = 3;
String result = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    case 4 -> "Thursday";
    case 5 -> "Friday";
    case 6 -> "Saturday";
    case 7 -> "Sunday";
    default -> "Unknown";
};
```

Switch

```
...
double value = switch (operation) {
    case "+" -> a + b;  // valor devolvido
    case "-" -> a - b;
    case "*" -> a * b;
    case "/" -> {
        assert b != 0: "Division by zero";
        yield a / b; // valor devolvido pelo bloco de código
    }
    default -> {
        assert false: "Unknown operation: " + operation;
        yield Double.NaN;
    }
};
```

- O comando `assert` gera uma exceção caso a condição não se verifique

CICLOS

Ciclos

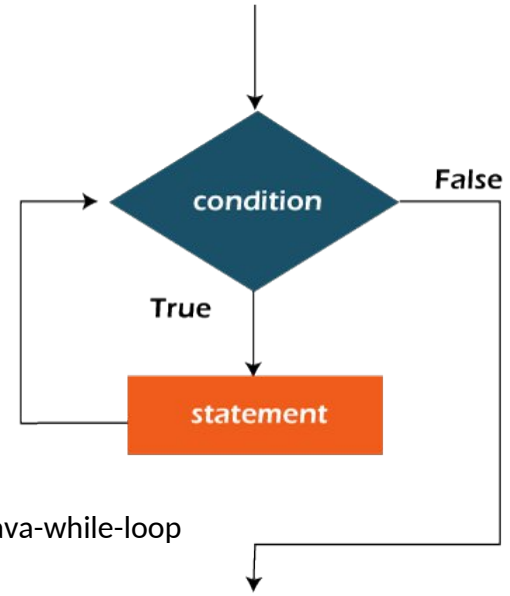
- Os ciclos servem para executar uma instrução, ou um bloco de instruções, várias vezes (iterações)
- Em cada iteração, é avaliada uma condição (expressão booleana)
- Quando a condição representar o valor lógico `false`, o ciclo termina

Source: https://www.javatpoint.com/java-while-loop

Ciclos

- Comando `while`
- Usado quando não se sabe quantas iterações terá o ciclo
- A condição é avaliada no inicio
- A instrução pode nunca ser executada, caso a condição tenha valor `false` na primeira iteração

```
 while (true){
     System.out.print("LOOP infinito!");
}
```

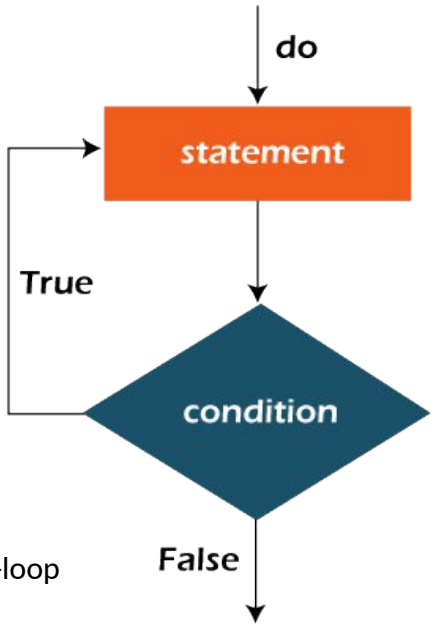


Source: https://www.javatpoint.com/java-while-loop

Ciclos

- Comando `do… while`
- Usado quando não se sabe quantas iterações terá o ciclo
- A expressão é avaliada no fim
- A instrução é sempre executada pelo menos uma vez

```
do {
    System.out.print("LOOP infinito!");
} while (true)
```



Source: https://www.javatpoint.com/java-do-while-loop

Ciclos

- Comando `for`
- Usado quando se deseja executar um código um determinado número de vezes

```
for (int i = 1; i <= 10; i++)    //10 iterações
    System.out.printf("%d ", i);   //Será impresso de 1 até 10
```

- Equivalente a um ciclo while

```
int i = 1;
while (i <= 10) {              //10 iterações
    System.out.printf("%d ", i); //Será impresso de 1 até 10
    i++;
}
```

Ciclos

- Controle de fluxo: facilitar o uso de ciclos
- `break`: sair imediatamente do ciclo
- Pode ser executado dentro de um `while`, `for`, `do...while`, ou `switch`

```
 for (int i = 1; i <= 10; i++) { //O loop é executado 5 vezes
    System.out.printf("%d ", i);
    if (i == 5)
        break;
} //Será impresso de 1 até 5
```

Ciclos

- Controle de fluxo: facilitar o uso de *loops*
- `continue`: serve para encerrar a execução de comandos
- Pode ser executado dentro de um `while`, `for`, `do...while`, ou `switch`

```
for (int i = 1; i <= 10; i++) { //O loop é executado 10 vezes
   if (i % 2 == 0)
       continue;
   System.out.printf("%d ", i);
} //Serão impressos os números ímpares entre 1 e 10
```

MÉTODOS

Métodos

- Métodos são blocos de código que pertencem a uma classe e tem por finalidade realizar uma tarefa
  - O conceito de classe será definido mais à frente
- Análogos às funções em C
- Cada método tem, no mínimo, a seguinte composição:

```
<tipo de dado que devolve> nomeDoMétodo (lista de parâmetros) {
        <instruções>
}
```

Métodos

- Um parâmetro (ou argumento) representa um valor fornecido ao método
- Um método pode não possuir parâmetros ou possuir vários parâmetros
- Cada parâmetro deve ser acompanhado do respetivo tipo

Métodos

- Um método pode devolver um valor
- O tipo do valor de devolvido é definido antes do nome do método
- O método devolve um valor através do comando `return`
- A palavra-chave **`void`** indica a ausência de um tipo de dado e é utilizado, por exemplo, para indicar que o método não devolve qualquer tipo de dado
- Exemplo: metodo `main`

```
public static void main(String[] args){ }
```

Métodos

- Exemplo: Operações aritméticas

```
class OperacoesAritmeticas {
   private static int somar(int num1, int num2) {
       return num1 + num2;
   }
   private static int subtrair(int num1, int num2) {
       return num1 - num2;
   }
   private static int multiplicar(int num1, int num2) {
       return num1 * num2;
   }
   private static int dividir(int num1, int num2) {
       return num1 / num2;
   }
}
```

Métodos

- Exemplo: Operações matemáticas

```
class OperacoesAritmeticas {
    public static void main(String args[]) {
        int x = 10, y = 3;
        System.out.printf("%d + %d = %d\n", x, y, somar(x, y));
        System.out.printf("%d - %d = %d\n", x, y, subtrair(x, y));
        System.out.printf("%d * %d = %d\n", x, y, multiplicar(x, y));
        System.out.printf("%d / %d = %d\n", x, y, dividir(x, y));
    }
}
```